HDOC-60

Anagram Checking Using Pointers and Character Frequency Counting

1. Problem Statement

Write a C program that reads two lowercase strings, s and t, and passes their addresses to a function named **check_anagram()**. The function must use pointers to determine whether both strings contain the same characters with the same frequencies. The order of characters is irrelevant. Strings of different lengths are not anagrams. Print "Anagram" for valid anagrams and "Not Anagram" otherwise. Use a character-frequency-counting approach.

2. Algorithm / Logic

1. Input two lowercase strings s and t.
2. Pass the addresses of s and t to check_anagram() using character pointers.
3. Find the lengths of both strings. If the lengths are different, print "Not Anagram" and return.
4. Create an integer frequency array of size 26 and initialize all elements to 0.
5. Traverse the first string using a pointer and increment the frequency of each character.
6. Traverse the second string using a pointer and decrement the frequency of each character.
7. Check all 26 frequency values. If every value is 0, both strings have identical character frequencies and are anagrams.
8. If any frequency value is non-zero, print "Not Anagram".

3. C Program

```
#include <stdio.h>
#include <string.h>

void check_anagram(char *s, char *t)
{
    int freq[26] = {0};
    char *p, *q;

    if (strlen(s) != strlen(t))
    {
        printf("Not Anagram\n");
        return;
    }

    p = s;
    while (*p != '\0')
    {
        freq[*p - 'a']++;
        p++;
    }

    q = t;
    while (*q != '\0')
    {
        freq[*q - 'a']--;
        q++;
    }
```

```
    for (int i = 0; i < 26; i++)
    {
        if (freq[i] != 0)
        {
            printf("Not Anagram\n");
            return;
        }
    }

    printf("Anagram\n");
}

int main()
{
    char s[100], t[100];

    printf("Enter first string: ");
    scanf("%99s", s);

    printf("Enter second string: ");
    scanf("%99s", t);

    check_anagram(s, t);

    return 0;
}
```

4. Explanation

The function check_anagram() receives the two strings as char pointers. Since an array name represents the address of its first element, calling check_anagram(s, t) passes the addresses of both strings.

The frequency array freq[26] stores the count difference for letters a to z. While traversing s, the program increments the corresponding frequency. While traversing t, it decrements it. Pointer variables p and q move through the strings one character at a time.

If the strings are anagrams, every increment caused by the first string is cancelled by a matching decrement from the second string, so all 26 values become zero. A length check is performed first because strings of different lengths cannot be anagrams.

5. Test Cases

Test	String s	String t	Expected Output	Reason
1	listen	silent	Anagram	Same characters with same frequencies.
2	triangle	integral	Anagram	Order differs, but frequencies match.
3	hello	world	Not Anagram	Character frequencies are different.
4	abc	ab	Not Anagram	Lengths are different.
5	aabbcc	abcabc	Anagram	Each character occurs twice in both strings.

6. Sample Output

```
Enter first string: listen
Enter second string: silent
Anagram
```

7. Result

The C program successfully checks whether two lowercase strings are anagrams using pointers and a character-frequency-counting approach. It correctly ignores character order, compares frequencies, rejects strings of different lengths, and prints the required result.